TECHNICAL REPORT · V0.2.17 · NOVEMBER 2026

Jinbao: A Local-First Desktop Game Coach

Turning a game client's own telemetry into one actionable sentence — and what breaks when you try

Yuan Cao

Jinbao Project · github.com/YuanCao1996/dsh-game-jinbao · MIT License

ABSTRACT

Jinbao is a Windows desktop coach that reads a game client's local telemetry — log streams and the LCU API — projects it into a small set of *facts*, decides whether the current moment is worth interrupting the player, and only then asks a language model for one or two actionable sentences, which it renders in an always-on-top overlay and optionally speaks. The engineering problem is not "call a model with game state". It is deciding when to speak, since a coach that narrates every event is worse than silence; not lying, since speculation must never be described as fact and optical observation must never guess a choice into the permanent record; and staying correct across time, since advice generated forty seconds ago may be actively harmful now. We describe the architecture, the three subsystems where most of the difficulty lives — trigger selection, augment observation, and advice invalidation — the privacy and update-integrity model, and three production failures that shaped the design.

本文记录这套系统的架构与取舍：事实投影、播报时机与准入控制、强化识别的确认式策略、 建议有效性与依赖指纹、签名更新与本地优先的数据模型，以及三个真实故障——它们比任何功能都更能 说明「让模型看懂游戏」难在哪里。

1 Introduction

Players already have statistics: win rates, tier lists, build sites. The questions that actually stop a player mid-match are situational and short-lived: *that fight just went wrong, why? · I have 1200 gold and two magic-damage threats — what do I buy now? · three augments are offered; which one fits the two I already own?* Each answer is worthless twenty seconds late, must name concrete entities — a champion, an item, an augment — and must be executable.

That framing produces four hard constraints (Table 1), and they in turn produce the architecture in §3. This report is deliberately organised around what broke rather than what shipped: three production failures in §8 explain more about the problem than any feature list.

Table 1. Constraints derived from the framing, and their consequences.

Constraint	Consequence
Advice expires	Every recommendation carries explicit dependencies and a TTL
Advice must be grounded	Facts and speculation are separated inside the prompt
Interruptions are costly	One admission gate decides what may speak
Data is intimate	All match data stays local; exports are redacted

2 Related Work

Overlay companions. Overwolf-class tools and services such as Blitz and Mobalytics read the same public surface — client logs and the Live Client Data API — and present statistics, build recommendations and post-game analysis. Their unit of output is a panel: many numbers the player must interpret in the moment. Our unit of output is a decision, which changes almost everything downstream: what to observe, when to speak, and how to bound output length.

LLM agents with tools. Recent agent frameworks show that tool-using models can plan over structured state. Applied to a live match, the binding constraints are latency, cost and trust rather than capability: an answer that arrives after the fight is worthless, and an answer that is confidently wrong is worse than silence. We therefore treat the model as a natural-language renderer over facts that the system already trusts, and we spend most of the engineering budget on *when* to call it and on constraining what it may assert.

Human-in-the-loop perception. Whenever a fact is derived from pixels rather than from an API — augment identity — we prefer an explicit confirmation step over an inferred guess. This is a deliberate precision-over-recall trade (§4.2), and it is the opposite of how most game-vision tools behave.

3 System Overview

Four layers, one direction of data flow (Figure 1). Acquisition produces an append-only event journal; projection is a pure function from journal plus snapshot to a view model; coaching decides whether to speak and what to say; presentation renders and optionally speaks it. Table 2 lists the invariant each layer must preserve.

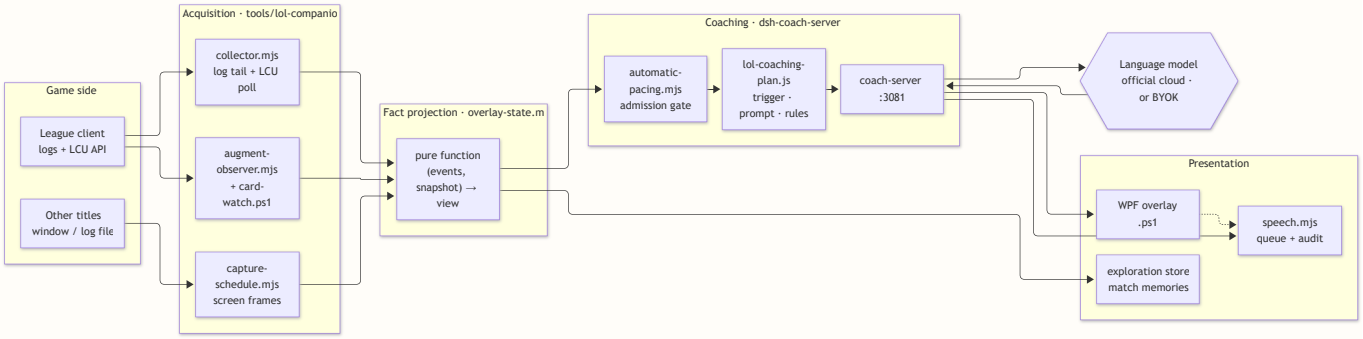


Figure 1. System context. Acquisition never mutates history; projection is pure; the coaching layer owns every decision about interrupting the player.

Table 2. Component responsibilities and the invariant each layer must preserve.

Layer	Directory	Invariant
Acquisition	`tools/lol-companion/`	Observations are append-only; nothing is rewritten after the fact
Projection	`overlay-state.mjs`	Pure: same events in, same view out
Coaching	`tools/dsh-coach-server/`	Facts and speculation are separated *in the prompt*
Presentation	`tools/desktop-release/`	Overlay failure must never kill the match pipeline
Delivery	`tools/jinbao-launcher/`	Ship only what the signature verifies

4 Acquisition

4.1 Log streams and the LCU API

The League client writes a per-match event journal containing kills, items, objectives, phase changes and raw snapshots. A collector tails it and publishes normalized events; a second stream polls the LCU for authoritative player state. Both are append-only and timestamped, so replay and live polling share one projection — which is what made the regression in §8.3 reproducible offline.

4.2 Augment observation

Augment identity is not exposed by the API, so it must be read from three card regions on screen. A wrong guess is *permanent*: it enters the match record and therefore every later recommendation. We render observation as an explicit state machine (Figure 2) and trade recall for precision under five guards (Table 3).

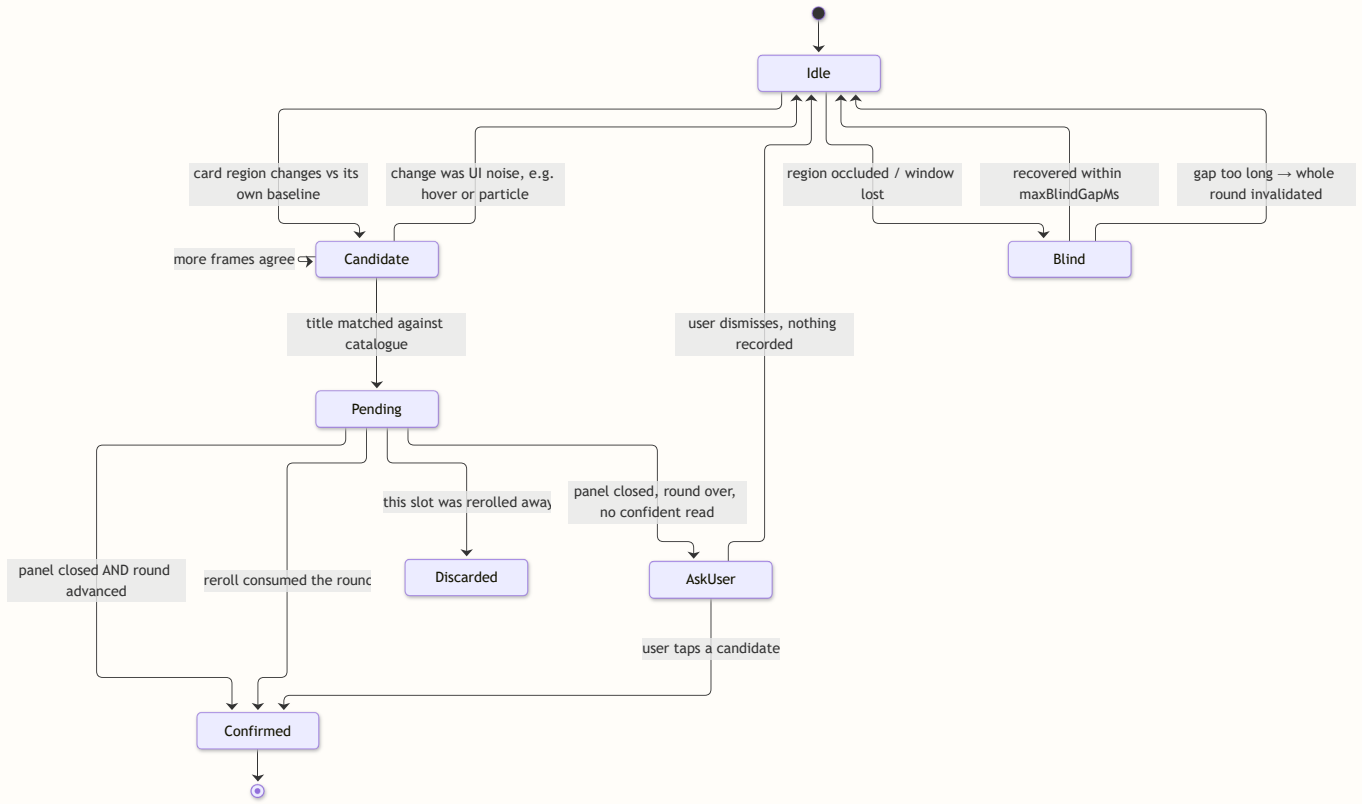


Figure 2. Augment observation as a state machine. A pending pick is settled by evidence — panel close, round change, reroll — never by timeout optimism.

Table 3. Guards around augment observation.

Guard	Value / effect
Frame freshness	≤ 4 s; a stale frame cannot confirm a live round
Blind-gap tolerance	15 s; beyond this the round is invalidated
Punctuation normalization	OCR read `质变．棱彩阶` vs catalogue `质变：棱彩阶` — a card silently vanished without it
Title joining	Adjacent lines in the same card only; descriptions are never fuzzy-matched
Confirmation fallback	User picks from this round's candidates; nothing is inferred

5 Fact Projection

`overlay-state.mjs` is a pure function of (events, snapshots) to a view model. The displayed clock comes from the game's elapsed seconds, never from observation time — mixing the two makes advice say "at 12:30" about a 09:10 event. Projected state is bounded: forty key-event rows and two hundred panel rows, so a long match cannot grow prompt size without limit.

> **Boundary.** The view is presentation-shaped: `events` and `augments` are rendered text, not arrays. That boundary is useful, and it is also the source of the worst regression in this project (§8.3).

6 Coaching Decision Layer

6.1 Speaking only when it changes a decision

Reasons are computed from deltas rather than from event presence, each with its own cooldown: death 15 s, material change 75 s, enemy change 30 s. Ordinary minion kills are context, not triggers. Figure 3 shows one turn; the upper branch — no reason, or inside a cooldown — is the common case by design.

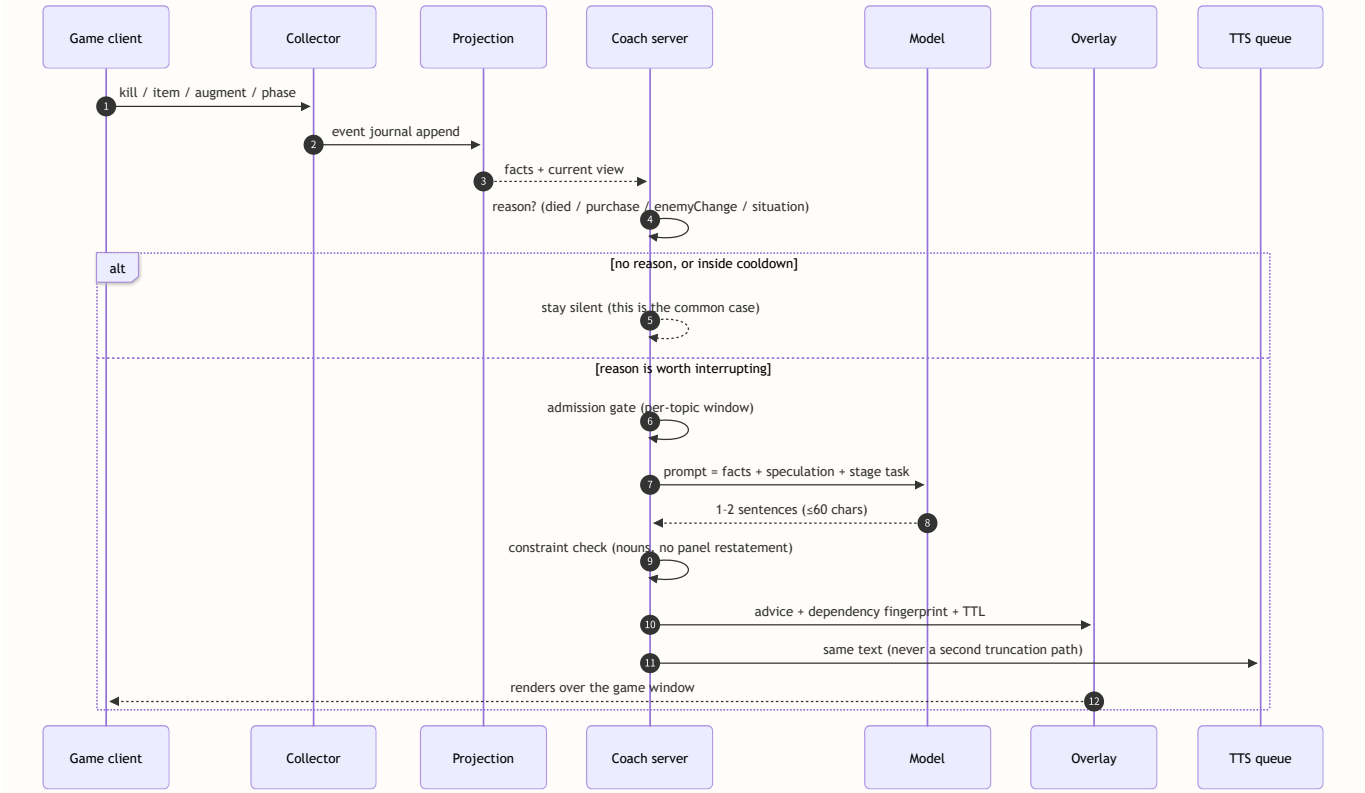


Figure 3. One advice turn. Most events produce nothing, deliberately.

6.2 Admission control

Battle advice and background build analysis share one permit but track *per-topic* windows. An earlier single shared window let frequent background tasks starve advice: we measured 20 `output_backpressure` skips in twelve minutes with `advicePublished: 0`.

6.3 Prompt construction and output bounds

Speculation is labelled *in the prompt*: unconfirmed augments are passed as candidates and may never be described as owned. Multikill counts come from a single field derived from the game's `KillStreak`, because models reliably miscount kills. The interface already shows rankings, so the answer must be a decision rather than a restatement, and a death turn must name one purchasable item. Output is at most two sentences and sixty characters; speech and display share the same source text, after a separate speech-truncation path was caught cutting mid-item ("亡者的板甲，针").

6.4 Validity by dependency fingerprint

Advice is not valid until a timer expires. Each recommendation records what it depends on, and validity is recomputed by comparing fingerprints (Figure 4, Table 4). A confirmed augment revokes augment-scoped advice; buying one component does not revoke objective advice, which depends on who is alive.

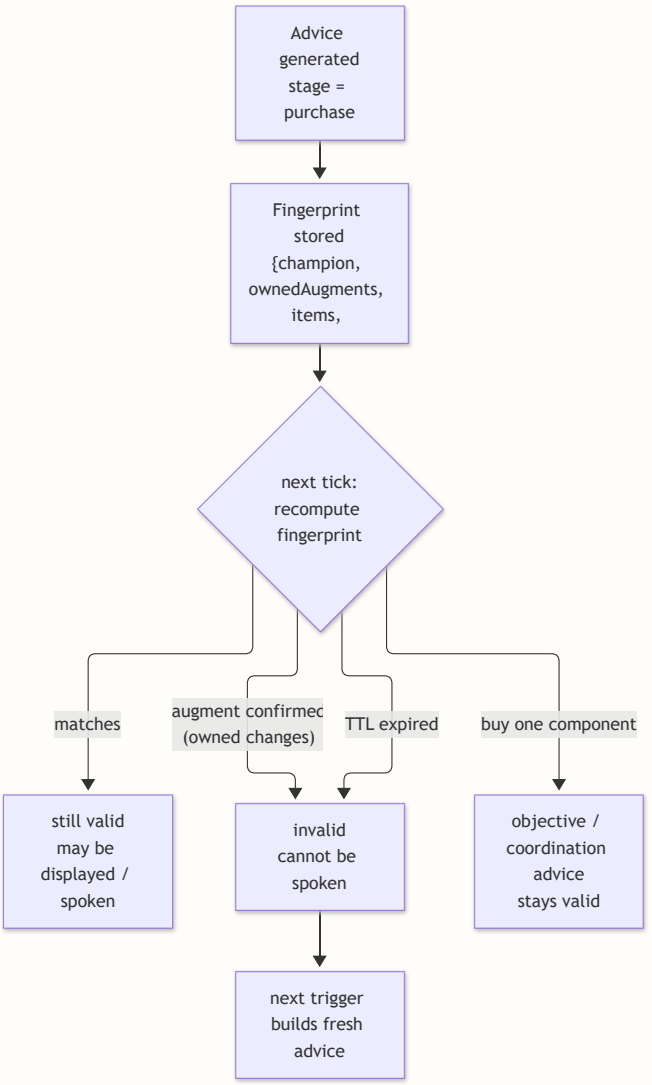


Figure 4. Invalidation by dependency fingerprint.

Table 4. Dependencies and lifetimes per stage.

Stage	Depends on	TTL
`opening`	champion **and owned augments**	10 min
`augment`	champion, owned, current offers	90 s
`postAugment`	champion, owned	90 s
`death` / `purchase`	champion, phase, level band, items, owned, enemy items	45 s
objective-class	champion, phase, alive allies, alive enemies	45 s

7 Case Study: Augments That Name Items

Some augments reference a specific item. 钢化你心 requires holding 心之钢 and stacking it; 命运之武器 applies to the first legendary item; 易损 rewards critical-strike items. In a recorded match the player confirmed 钢化你心 at roughly three minutes, yet the six-item plan read *Mercury's Treads* → *Force of Nature* → *Randuin's* → *Frozen Heart* → *Spirit Visage* → *Stoneplate* — no Heartsteel — while a later recommendation told the player

to build it. The opening plan declared its dependency as the champion alone:

```
if (stage === 'opening') return { champion: me.champion };  // augments ignored
```

With a ten-minute TTL that plan stayed valid for the rest of the early game. The fix has three parts: opening plans depend on owned augments; a hard prompt rule requires that an augment which names an item forces it into the build order with the reason stated; and `postAugment` re-derives the remaining two or three items at the moment the augment becomes known. An audit of the last eight matches found four observed augments constrain item choice, so this is not an edge case.

8 Failure Modes and Lessons

Table 5. Failures, symptoms, and the rule adopted afterwards.

Failure	Symptom	Rule adopted
JavaScript comment in a PowerShell file	Overlay crashed on every collapse; the supervisor restarted it, which looked like "the window disappears and comes back"	`.ps1` files are UTF-8 with BOM and use only `#` comments; every unhandled exception logs source, line and code
Log statement between `elseif` and `else`	`CommandNotFoundException: else` at a known line	Instrument first, then edit; the next bug after a fix is usually the fix itself
Display string treated as data	`(view.events || []).map(…)` threw; `try/catch` swallowed it; three consecutive matches recorded no memory	Never silence a pipeline: log the skip reason or fail loudly

The third case is the most instructive. The projection boundary is real — `events` is text for rendering, not an array — and the recovery path is now uniform: the recorder reads the match's own journal for authoritative `GameEnd.Result`, final KDA, level, duration and multikill streaks (Figure 5), and a replay restored the lost matches from those journals.

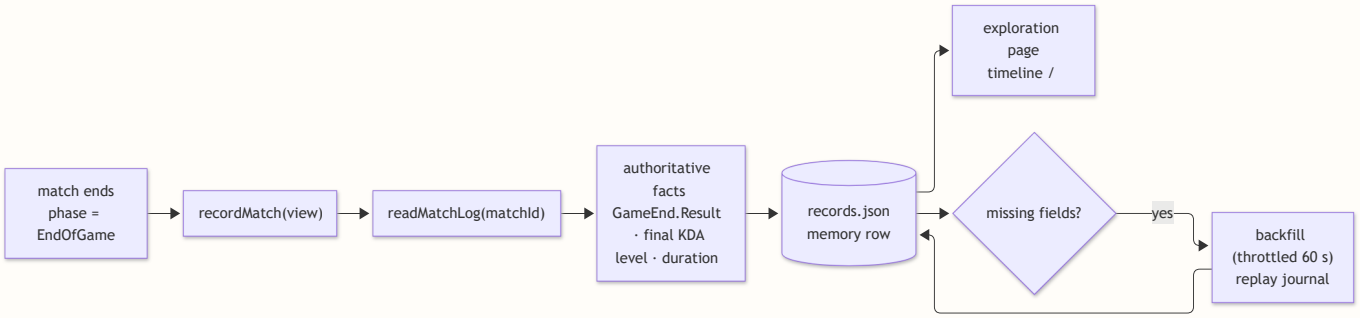


Figure 5. The memory pipeline that replaced the broken path. Authoritative values come from the journal; a throttled backfill repairs records missing fields.

9 Presentation and Voice

The overlay is WPF hosted by PowerShell in four display modes (Figure 6). The narrow mode keeps a live subtitle strip rather than collapsing to a dot, so the last sentence is never lost. Subtitle splitting respects semantics: punctuation never starts a line, tokens such as `No.22`, `53.5%` and `15/6/16` stay whole, and at most two rows are visible.

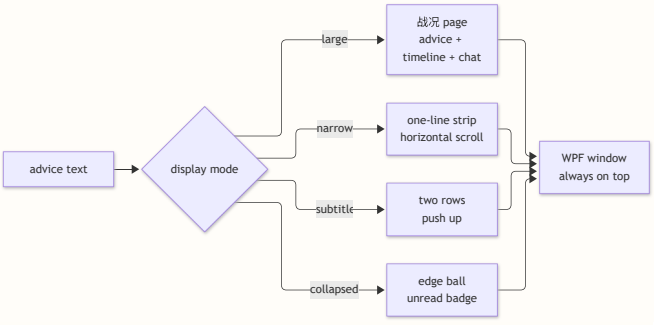


Figure 6. Display modes and what each preserves.

The speech queue expires *only queued* items; once audio starts it finishes. Every transition — `queued`, `started`, `completed`, `interrupted`, `discarded` — is appended to a local audit log with the original and spoken text, so display-versus-speech drift is auditable rather than argued about.



Figure 7. A real advice stream: death, coordination and build items, one or two sentences each.

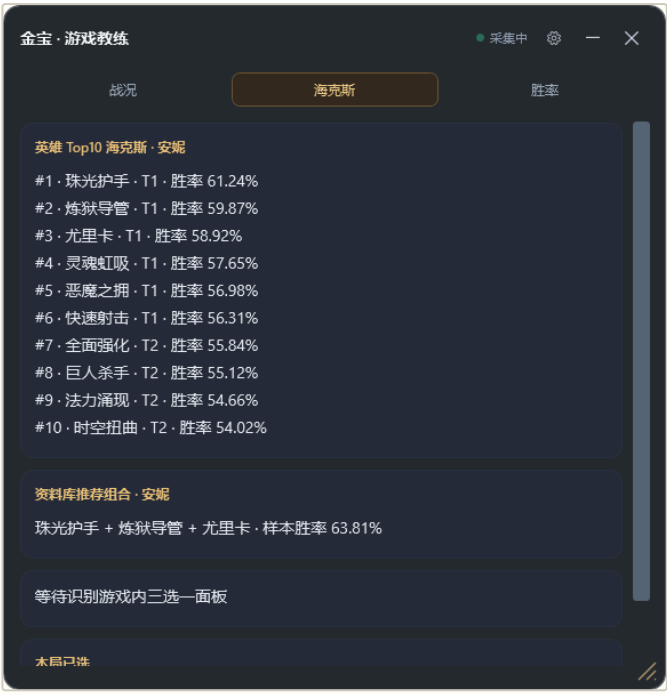


Figure 8. Augment ranking for the current champion, owned picks included.

10 Privacy and Update Integrity

Match data, memories, preferences and diagnostics live under the user's profile directory and are never uploaded; there is no leaderboard. Credentials are DPAPI-encrypted per Windows user, and bring-your-own-key model secrets never reach our servers. Diagnostics exports are redacted: no chat text, player names, credentials or screenshots.

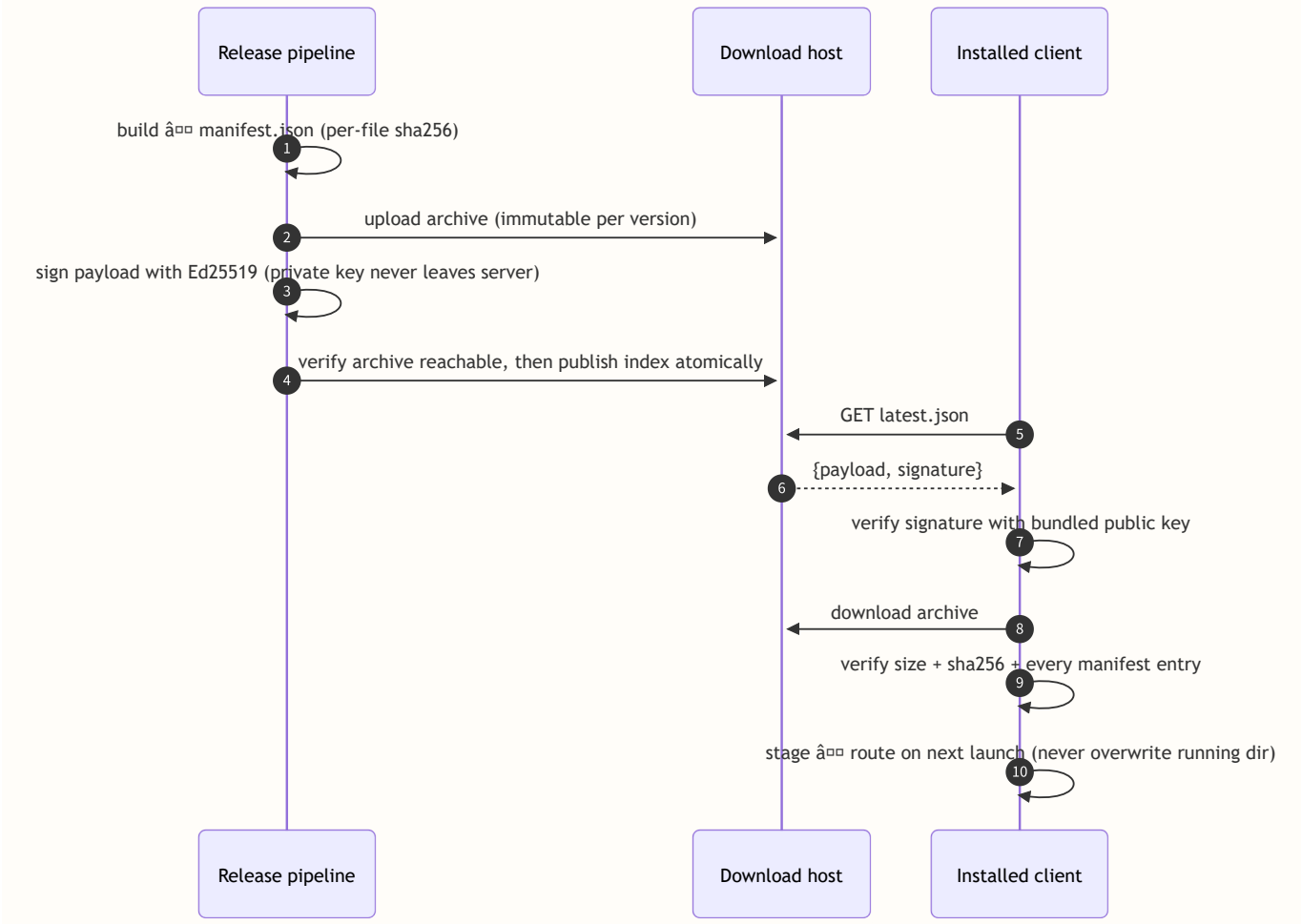


Figure 9. Signed update pipeline. The index is published only after the archive is verified reachable, so a client can never accept an index pointing at a missing artifact.

Table 6. Update integrity controls.

Control	Implementation
Tamper resistance	Ed25519 over the exact payload bytes; fixed origin; public key in client
Rollback safety	Immutable version paths; atomic index publication
Partial download	Size and per-file hashes against the package manifest
Failure policy	Any mismatch keeps the old version; updates stage and route on next launch

11 Evaluation and Limitations

Table 7. Verified in this release cycle.

Area	Evidence
Advice validity	Fingerprint tests: a later augment invalidates; a component purchase does not
Augment-aware builds	Prompt assertions plus offline replay of a real journal
Memory recording	Replay produced KDA 32/11/21, win, level 18, 1247 s, multikills
Speech audit	116 recorded transitions in one match, diffable
Overlay stability	Zero unhandled UI exceptions across collapse and resize cycles

Limitations. Augment identity is OCR-derived; ambiguous rounds fall back to confirmation, and recall is intentionally better than precision. League of Legends is the only deeply adapted title — other games receive window observation with explicitly unverified identity. The model can still produce advice that is technically true but operationally vague; rules reduce this without eliminating

it. Some sandbox-dependent tests are environment-sensitive and excluded from the default run. Advice quality has not been validated against a control group: "good advice" remains a judgement call, not a measurement.

Non-goals. Predicting match outcomes, assigning probabilities to a live game, automating input, reading game memory, and building a cloud dashboard or leaderboard.

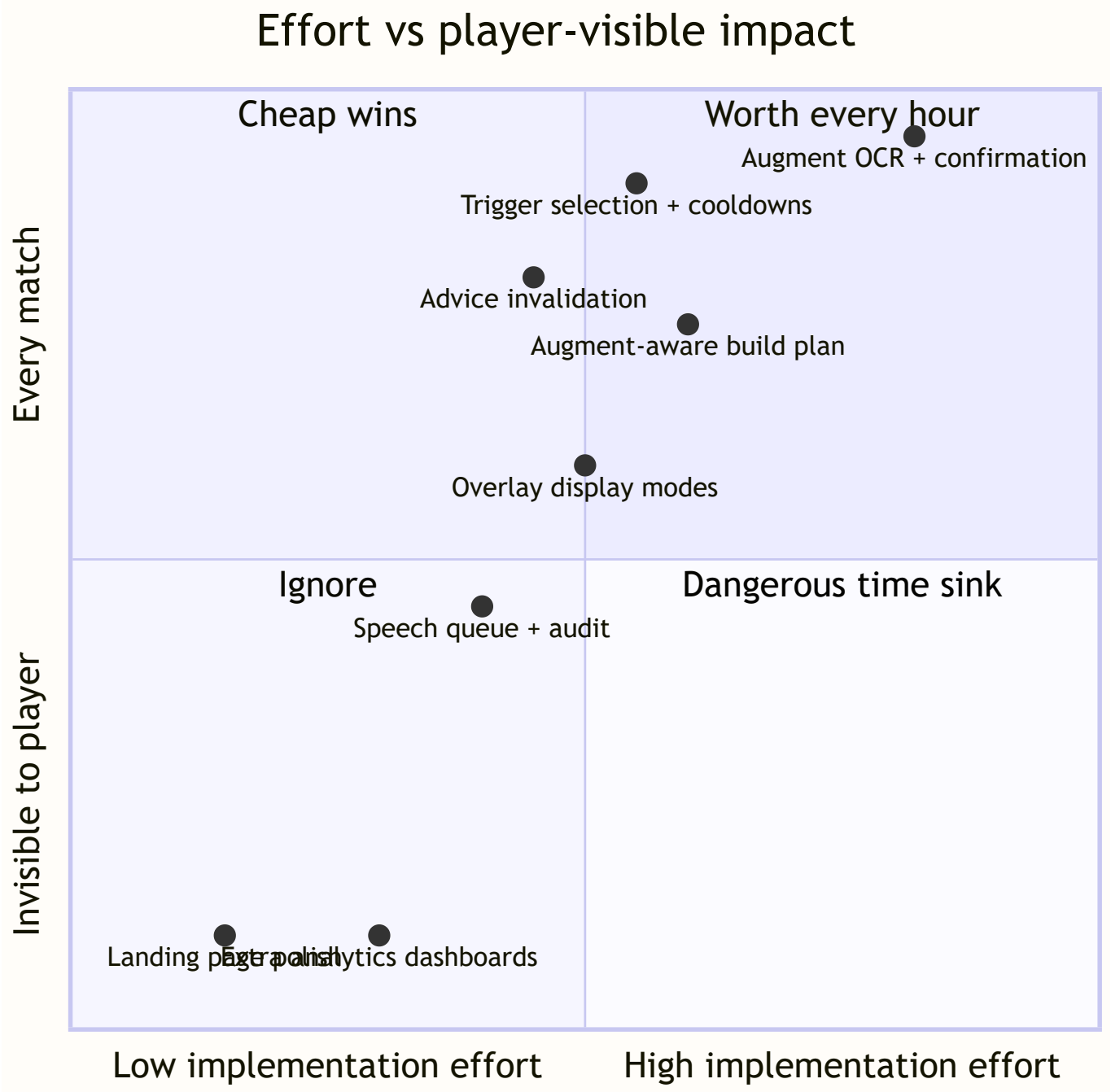


Figure 10. Where effort pays. The upper-left cluster — cheap to describe, expensive to get right — is the reason this project exists.

12 Conclusion

A local-first coach is mostly an exercise in restraint: read only what the client already publishes, speak only when the answer changes a decision, label every piece of speculation, and make invalidation explicit rather than hopeful. The hard parts are perceptual precision and temporal correctness, not language generation — which is why most of this report is about failure modes.

References

1. Riot Games. *League of Legends Live Client Data API*. Developer documentation.
2. Riot Games. *League Client Update (LCU) API* — local REST surface of the desktop client.

3. B. Josefsson, I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, 2017.
4. Microsoft. *Windows Data Protection API (DPAPI)*.
5. Microsoft. *Windows Presentation Foundation*.
6. OpenJS Foundation. *Node.js 22 LTS*.
7. Mermaid diagram tooling — figures rendered from the repository's Mermaid sources.
8. Source: github.com/YuanCao1996/dsh-game-jinbao (`docs/ARCHITECTURE.md`, `docs/TECHNICAL_REPORT.md`).

Jinbao · Technical Report v0.2.17 · MIT License · Corrections via GitHub issues are welcome